

데이터 추출하기



데이터 추출하기

- 원하는 구간, 조건에 해당하는 데이터를 추출해야 할 때 사용
- 데이터 분석 결과를 얻기 위해, 적절한 데이터를 선택하고 추출하기 위해 필요한 과정



Pandas 논리연산자

- and → &

```
(A > 30) & (A < 50)
```

- or → |

- not → ~

- 소괄호 ()를 사용하여 우선 순위 명시



```
(30 < A < 50)
```

```
(A > 30) | (B < 50)
```

조건에 따른 인덱싱: Boolean indexing

비교 연산자나 논리 연산자를 사용하여 데이터프레임에서 조건에 맞는 행 추출 가능

df[조건식]

df[df["어른"]>1000]



	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일
0	2016-01-01	O	구름 조금	3359	2799	141.0	419	47	0	1023	4382	2016	1	1	금
1	2016-01-02	O	구름 많음	5173	4370	203.0	600	100	111	2092	7265	2016	1	2	토
2	2016-01-03	O	구름 많음	3008	2571	128.0	309	91	0	1549	4557	2016	1	3	일
8	2016-01-09	O	구름 많음	1418	1227	78.0	113	47	80	1222	2640	2016	1	9	토
9	2016-01-10	O	구름 많음	2211	1839	96.0	276	34	61	1393	3604	2016	1	10	일
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
1077	2019-03-23	O	눈/비	2097	1760	44.0	293	42	0	1465	3562	2019	3	23	토
1078	2019-03-24	O	맑음	10039	8502	161.0	1376	74	35	2748	12787	2019	3	24	일
1083	2019-03-29	X	구름 조금	1644	1447	120.0	77	14	188	1226	2870	2019	3	29	금
1084	2019-03-30	O	흐림	1539	1326	44.0	169	29	115	913	2452	2019	3	30	토
1085	2019-03-31	O	구름 조금	3061	2563	111.0	387	53	0	1357	4418	2019	3	31	일

df의 "어른" 컬럼의 값이 1000보다 크다 라는 조건식을 만족하는 데이터를 추출

복잡한 조건을 활용한 Boolean indexing

pandas 논리연산자를 활용하면 복잡한 조건을 추가할 수 있음

```
df[(df["어른"]>1000) & (df["어린이"]>1000)]
```



	날짜	공휴일	날씨	유료합계	어른	청소년	어린이	외국인	단체	무료합계	총계	연	월	일	요일
78	2016-03-19	O	구름 조금	26448	22735	422.0	2990	112	178	6683	33131	2016	3	19	토
79	2016-03-20	O	구름 조금	27093	22947	535.0	3370	176	0	7267	34360	2016	3	20	일
85	2016-03-26	O	맑음	16958	14114	408.0	2094	192	62	4790	21748	2016	3	26	토
86	2016-03-27	O	맑음	23610	20041	487.0	2860	205	60	6879	30489	2016	3	27	일
92	2016-04-02	O	구름 조금	31407	26605	561.0	3643	169	201	6380	37787	2016	4	2	토
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
937	2018-11-03	O	맑음	11573	9817	231.0	1525	152	436	5053	16626	2018	11	3	토
938	2018-11-04	O	구름 조금	12125	10223	307.0	1595	79	195	3778	15903	2018	11	4	일
1063	2019-03-09	O	NaN	12602	11023	157.0	1422	52	74	5231	17833	2019	3	9	토
1071	2019-03-17	O	NaN	12169	10313	199.0	1657	51	30	3549	15718	2019	3	17	일
1078	2019-03-24	O	맑음	10039	8502	161.0	1376	74	35	2748	12787	2019	3	24	일

df의 "어른" 컬럼의 값이 1000보다 크고 "어린이"컬럼의 값이 1000보다 크다 라는
조건식을 만족하는 데이터를 추출



여러 값 중 하나인 행 추출: isin()

컬럼 값이 지정한 목록 중 하나에 해당하는 행을 추출

```
df[df["컬럼명"].isin([값1, 값2, ...])]
```

```
# isin 없이 (or로 일일이)
```

```
df[(df["날씨"] == "맑음") | (df["날씨"] == "흐림")]
```

```
# isin 사용 (같은 결과, 깔끔)
```

```
df[df["날씨"].isin(["맑음", "흐림"])]
```

- 컬럼 값이 **리스트 안의 값 중 하나라도 일치하는** 행을 추출
- 같은 컬럼에 == 를 or(|)로 여러 번 묶는 걸 **한 줄로** 대체
- 비교할 값들을 **리스트([])**로 전달

조건 추출을 더 간결하게 : query()

Boolean indexing과 같은 결과를, 조건을 문자열로 써서 더 읽기 쉽게 작성하는 방법

```
df.query("조건식")
```

```
df.query("컬럼명1 > 값")
```

단일 조건

```
df.query("컬럼명1 > 값 and 컬럼명2 < 값")
```

여러 조건 (and / or)

```
df.query("컬럼명1 == '문자값'")
```

문자열 비교 (안쪽은 작은따옴표)

조건을 문자열로 써서 행 추출

컬럼명을 이름 그대로 사용

여러 조건은 **and / or** 로 연결



라벨을 활용한 데이터 추출: loc

- 라벨을 사용하여 데이터프레임에서 데이터를 추출하는 방법
- 인덱스이름과 컬럼이름을 입력 받아 그에 해당하는 데이터 추출

```
df.loc["인덱스이름", "컬럼이름"]
```

index가 456인 행의
"총계" 열 데이터 추출

```
df.loc[456, "총계"]
```

338

index가 24, 67, 126인 행의
"공휴일", "어른", "어린이" 열 데이터 추출

```
df.loc[[24, 67, 126], ["공휴일", "어른", "어린이"]]
```

	공휴일	어른	어린이
24	X	58	4
67	X	465	8
126	O	4764	1031

index가 35부터 40까지인 행의
전체 열 데이터 추출

```
df.loc[35:40, :]
```

	날짜	공휴일	날씨	유료함게	어른	청소년	어린이	외국인	단체	무료함게	총계	연	월	일	요일	매출액
35	2016-02-05	X	구름 조금	304	234	41.0	29	14	0	536	840	2016	2	5	금	3040000
36	2016-02-06	O	맑음	1082	924	53.0	105	40	0	976	2058	2016	2	6	토	10820000
37	2016-02-07	O	맑음	1452	1251	74.0	127	191	0	676	2128	2016	2	7	일	14520000
38	2016-02-08	O	구름 많음	2654	2286	123.0	245	429	0	567	3221	2016	2	8	월	26540000
39	2016-02-09	O	맑음	5453	4610	223.0	620	209	0	1607	7060	2016	2	9	화	54530000
40	2016-02-10	O	맑음	7080	6037	251.0	792	141	0	2869	9949	2016	2	10	수	70800000



순서를 활용한 데이터 추출: iloc

- 행과 열의 위치(순서)를 이용해 데이터를 추출할 때 사용
- 행이나 열의 위치(순서)를 나타내는 정수를 입력 받아 해당 데이터 추출

```
df.iloc[행의 위치, 열의 위치]
```

```
df.iloc[42, 5]
```

17.0

```
df.iloc[3:7, 4:7]
```

	어른	청소년	어린이
3	602	NaN	235
4	319	35.0	62
5	277	41.0	49
6	250	23.0	36

loc vs iloc

loc은 index 이름을 활용, iloc은 정수형 위치(순서)를 활용하여 인덱싱

index	국어	수학	영어
민수	95	88	96
도윤	72	72	68
서아	83	93	85
연우	89	80	94
하준	100	95	93

`grade.loc["연우", "수학"]`

`grade.iloc[3, 1]`



loc과 iloc을 활용한 값 변환

- loc과 iloc을 활용하여 특정 원소 값 변환 가능
- loc과 iloc을 활용해 바꿀 위치를 지정하고 해당 위치에 넣을 값을 지정

index	국어	수학	영어
민수	95	88	96
도윤	72	72	68
서아	83	93	85
연우	89	80	94
하준	100	95	93

```
df.loc["서아", "국어"] = 100
```

```
df.iloc[1, 1] = 99
```



index	국어	수학	영어
민수	95	88	96
도윤	72	99	68
서아	100	93	85
연우	89	80	94
하준	100	95	93